



Transcript

Week 9: Unsupervised Learning

Video 9.1 - Unsupervised Learning (01:10)

What if your data could reveal hidden patterns without you even knowing what to look for?

Unsupervised learning unlocks the power of finding structure in chaos, helping machines uncover insights without predefined labels.

Unsupervised learning helps uncover hidden patterns in data without labeled outputs. This module introduces the differences between supervised and unsupervised learning, explaining when to use each. Key techniques like clustering and dimensionality reduction will be explored to simplify complex data. Practical implementation using Python libraries like Scikit-learn will demonstrate how to apply these methods. Finally, you'll learn to analyze, visualise, and interpret results to make meaningful insights from your models.

Let's get started!

Video 9.2a - How Machines Learn Patterns Without Labels (06:47)

Did you know that machines can uncover hidden patterns in data—without any labels or human guidance?

Imagine you have a box of mixed fruits, but you don't know their names. Even without labels, you can still sort them by size, shape, or colour. That's similar to how unsupervised learning works. It helps machines find patterns in data without needing predefined labels or instructions.

Unsupervised learning is a type of machine learning where the algorithm analyses data without predefined labels or outcomes. Unlike supervised learning, where the model has a target to predict, unsupervised learning identifies hidden patterns or structures within the data. It does this by recognising similarities and relationships among data points.

There are two main tasks in unsupervised learning—clustering and association. Clustering groups similar data points based on shared features, while association finds relationships or patterns between variables. For example, in marketing, unsupervised learning can group customers based on purchasing behaviour, even when predefined categories are unavailable.

Supervised and unsupervised learning are two key types of machine learning. In supervised learning, the algorithm is trained using labelled data, meaning the input has known outcomes. This allows the model to make predictions, such as estimating house prices or identifying spam emails.

In contrast, unsupervised learning deals with unlabelled data. The algorithm must analyse the structure of the data and uncover patterns on its own. This approach is useful for tasks like grouping similar products or identifying relationships within datasets.

The key difference is that supervised learning relies on labelled data, while unsupervised learning does not.



Unsupervised learning helps find hidden patterns in data. One key method is clustering, where similar data points are grouped based on common features. Businesses use this for customer segmentation, tailoring marketing strategies to different purchasing behaviours. Another method is association, which uncovers relationships between items. For example, market basket analysis shows that people who buy bread often buy butter too.

Unsupervised learning consists of different techniques, each serving a unique purpose.

First, **clustering** groups data into meaningful clusters based on similarities. It is useful when segmenting data into distinct categories without predefined labels. Common algorithms include **K-Means**, **Hierarchical Clustering**, and **DBSCAN**, which stands for Density-Based Spatial Clustering of Applications with Noise.

Next, **dimensionality reduction** helps simplify complex datasets by reducing the number of features while preserving key information. This is especially useful when working with high-dimensional data. Popular methods include **Principal Component Analysis, or PCA**, and **t-Distributed Stochastic Neighbour Embedding, or tSNE**.

Finally, **association rule learning** uncovers relationships between variables in large datasets. It is widely used in market basket analysis to identify patterns, such as customers who buy bread also purchasing butter. Common algorithms include **Apriori** and **Equivalence Class Transformation, or Eclat**.

Unsupervised learning is widely used in real-world applications.

One common use is customer segmentation, where businesses group customers based on purchasing behaviour. This helps improve marketing strategies and deliver personalised experiences.

Another key application is anomaly detection, which identifies unusual data patterns. This is useful in fraud detection, where deviations from normal behaviour can indicate suspicious activity.

Recommendation systems also rely on unsupervised learning to suggest content based on past viewing or purchasing habits. Streaming and e-commerce platforms use this to enhance user experience.

In healthcare, gene expression data analysis helps researchers identify biological patterns, advancing medical research.

One of the most important tasks in unsupervised learning is clustering. It helps uncover patterns in data without predefined labels. In this example, a two-dimensional plot shows how data points are grouped into clusters based on their similarities. Each color represents a different cluster, and the red "X" markers indicate the centroids of these clusters.

What makes this interesting is that the algorithm identifies these clusters without any prior instructions on how to group the data. It simply analyses the features and detects natural groupings. This demonstrates the power of unsupervised learning—discovering hidden patterns that might not be immediately visible to us.

Unsupervised learning is a powerful technique for discovering patterns in unlabeled data, but it comes with notable challenges. One of the primary difficulties is the lack of ground



truth, making it hard to evaluate model performance without predefined labels. Additionally, identifying complex patterns in data without guidance is challenging, often requiring careful tuning and interpretation. Another issue is interpretability, as the discovered patterns or clusters may not always be clear or actionable.

Despite these challenges, unsupervised learning is widely used in clustering and anomaly detection, allowing models to work without predefined categories. It is particularly useful in identifying hidden structures in data, such as customer segmentation or fraud detection. However, due to the absence of clear reference points, extracting meaningful insights can sometimes be difficult, making its application more complex compared to supervised learning.

Now, think about how you can leverage unsupervised learning to uncover hidden insights in your data?

Video 9.2b: How Machines Learn Patterns Without Labels contd.

What if data had no labels — just raw information? Unsupervised learning helps machines uncover patterns on their own. In this module, you'll explore how that works using clustering, scaling, and dimensionality reduction.

To begin our machine learning project, we first import the essential libraries—Pandas, Sigh-kit-learn, Mat-plot-lib, and Seaborn. Next, we use the load underscore iris function to load the Iris dataset. We store the input features in a Data Frame called X, and the corresponding target values in a Series named y. These variables form the foundation for both supervised and unsupervised learning tasks.

Now let's visualise the predictions. We start by creating a figure using plt dot figure, setting its size to ten by five. Next, we generate two side-by-side plots using plt dot subplot. Finally, we use plt dot tight underscore layout and plt dot show to neatly arrange and display both graphs.

Let's now visualise the difference using the Iris dataset. On the left, we see the result of a supervised learning method—Logistic Regression,. This model learns from labelled data to classify new inputs. On the right is an unsupervised learning method—K-Means clustering. It identifies patterns and groups the data without ever seeing the true labels. While it found clusters that resemble the real species, it did this based purely on feature similarity, not on pre-assigned labels. Both methods produce useful grouping. Supervised learning relies on known outcomes, whereas unsupervised learning explores the data with no prior label information.

Let's look at the types of features used in unsupervised learning. These models don't use labels — instead, they rely entirely on the input features to find patterns. Features can be numerical, like income or temperature, or categorical, like city or product type. Some are ordinal, meaning the order matters — like ratings from 'poor' to 'excellent'. Others may be binary, such as yes or no, or text-based, like reviews or comments. Unsupervised algorithms don't know which features are more important. They learn everything from the structure in the data. Since most algorithms require numerical input, we often need to convert text and categories into numbers using techniques like one-hot encoding or embeddings.



Let's start by creating a small dataset for unsupervised learning. We import Pandas for data handling, and modules from Sigh-kit-learn for preprocessing, pipelining, and clustering. Next, we use P-D dot Data Frame to build a dataset with four features: age, gender, income, and satisfaction. 'Age' and 'Income' are numerical features. 'Gender' and 'Satisfaction' are categorical—they'll need to be transformed before clustering. This dataset is now ready to move into the preprocessing and modelling phase.

Before we cluster the data, we need to prepare it properly. We start by splitting the features into two groups: numerical features like age and income, and categorical features like gender and satisfaction. Then, we build a preprocessing pipeline using Column Transformer. This lets us scale the numeric features using Standard Scaler and convert the categorical features into numbers using One-Hot-Encoder. This step is important because clustering algorithms work best when all input values are numerical and on a similar scale.

Now we apply both preprocessing and clustering in one go using a Pipeline. We combine the preprocessing step with K-Means, setting the number of clusters to two. Then, we run the entire sequence on the dataset using pipeline dot fit.

Finally, we assign the predicted cluster labels to the dataset using pipeline dot named underscore steps, which helps us retrieve the labels generated by the clustering step.

Here's the final output after applying the full pipeline. Each row represents a data entry with features like age, gender, income, and satisfaction. We've now added a new column called Cluster, which contains the results of the K-Means algorithm.

Each value in the Cluster column is either zero or one, showing which group the data point was assigned to. These groupings were created without using any labels — based only on how similar the data points are to one another.

Let's look at the table on screen. It compares the effects of scaling across three key areas.

First, distance-based models. Without scaling, features like income — which have large numerical ranges — can dominate the results. But with scaling, all features contribute equally to distance calculations.

Next, clustering quality. Without scaling, the algorithm may form biased or irrelevant groupings. With scaling, clusters become more natural and meaningful.

Finally, PCA directionality. Without scaling, the variance gets pulled toward features with large values. With scaling, PCA captures variance more evenly across all features.

At the bottom, you can see the algorithms most affected by scaling: K-Means, Hierarchical Clustering, D-B-SCAN, P-C-A, and t-S-N-E. Each of these relies on feature magnitude, so scaling helps ensure fair input from every variable.



Let's begin by creating our dataset.

We import Pandas, Num Pie, Mat-plot-lib, and modules from Sigh-kit-learn.

We then generate an unbalanced dataset using P-D dot DataFrame, with two features: Age, ranging from 20 to 60, and Income, ranging from twenty thousand to one lakh fifty thousand.

This difference in scale between Age and Income will help us demonstrate why data scaling is important before applying clustering.

Now we apply the K-Means algorithm with and without scaling.

First, we fit K-Means on the raw data. Since Income has a much larger range, it will dominate the clustering outcome.

Next, we apply Standard-Scaler to the data. This method scales all features so they have similar influence.

We then apply K-Means again — this time to the scaled dataset.

By comparing both outputs, we'll see how scaling affects the clustering results.

Let's now compare the clustering results.

On the left, we see the output from unscaled data — where Income dominates the clusters.

On the right, we see the result from scaled data — where both Age and Income are treated equally.

This side-by-side view clearly shows how scaling leads to more balanced and meaningful groupings.

It highlights why feature scaling is essential when working with distance-based models like K-Means.

This demo shows a powerful comparison.

We created synthetic data with two features — Age and Income.

In the plot on the left, we run K-Means without scaling. The result?

Clusters are distorted because Income, with its larger values, overpowers the model.

On the right, we apply Standard-Scaler to scale both features.

Now, Age and Income are on the same range, and the clusters look much more balanced and meaningful.

This highlights a critical point: Scaling is not optional — it's essential in unsupervised learning when features vary in scale.

In data science, dimensionality refers to the number of input features in your dataset.

Each feature adds a new axis to the data space.

So, if you have two features — like height and weight — your data lives on a two-dimensional plane.

Add a third feature, like age, and you move into three-dimensional space.

Modern datasets often have dozens or even hundreds of features. This is called high-dimensional data.

Every additional feature increases the number of dimensions — four becomes 4-D, five becomes 5-D, and so on — even though we can't visualise anything beyond 3-D.

Understanding dimensionality is important because as it increases, many machine learning algorithms struggle or behave differently.

That's why dimensionality reduction is such a critical topic in unsupervised learning — something we'll explore soon through techniques like P-C-A.

Why is dimensionality such a big deal in unsupervised learning?



Because of something called the Curse of Dimensionality.

As we add more features to a dataset, the data spreads out across more dimensions. That might sound useful — but it creates serious problems.

First, in high-dimensional space, distances become less meaningful. All points start to look equally far apart, which makes similarity-based methods like K-Means and D-B-SCAN unreliable.

Second, data becomes sparse. This makes clustering difficult, especially for density-based algorithms.

Third, the risk of overfitting increases. The model might start learning noise instead of real patterns.

And finally, the more features we have, the more computationally expensive it becomes to process the data.

That's why it's so important to reduce the number of dimensions before clustering — using methods like Principal Component Analysis or P-C-A.

Let's begin by generating high-dimensional data.

We use make_underscore_blobs to create a synthetic dataset with three clusters and fifty features.

Next, we apply Standard Scaler to ensure that all features have equal weight.

This standardisation step is crucial before applying PCA or clustering, especially when features are on different scales.

Now we apply P-C-A, or Principal Component Analysis, to reduce the dataset from fifty dimensions to just two.

This makes it possible to visualise high-dimensional patterns in a 2D space.

In the plot, each point represents a sample, and the colours indicate the original cluster assignments.

This shows how dimensionality reduction reveals structure in complex data — making it easier to interpret and use in unsupervised learning tasks.

In this demo, we generate a dataset with fifty features and three underlying clusters. This means the data lives in a space that's too complex for humans to visualise directly.

To make the structure easier to see, we apply Principal Component Analysis, or P-C-A, to reduce the data from fifty dimensions to just two.

The resulting scatter plot shows how the samples naturally group into clusters.

This visualisation helps us uncover the hidden structure within the high-dimensional space —

showing how dimensionality reduction is key to understanding and improving unsupervised learning results.

Let's look at customer segmentation — a real-world use case of unsupervised learning.

Customer segmentation means grouping people based on shared characteristics, such as age, location, income, or buying behaviour.

The goal is to find natural patterns in the data — not based on predefined labels. We don't tell the model what "Segment A" or "Segment B" is.



Instead, the algorithm learns to group similar customers by itself. That's what makes this unsupervised learning — because the model explores patterns on its own.

We use this to help businesses personalise their services — like targeting offers, recommending products, or improving customer experience — all based on the hidden segments the algorithm discovers.

Let's explore the features commonly used in customer segmentation and how each one contributes to understanding customer behaviour.

We start with age, which gives a basic sense of how old the customer is. This is often used alongside annual income, which adds economic context — helping us distinguish between different income groups.

Next, we have the spending score. This is a synthetic value, usually between zero and one hundred, that reflects spending habits — showing whether someone is more conservative or free-spending.

We then move on to region, which helps us identify geographic preferences — such as whether a customer lives in Delhi, Mumbai, or another area. And to complement that, membership type reveals loyalty levels — for example, Silver, Gold, or Platinum members — which can be a strong indicator of customer value.

Now, consider website visit count. This tells us how frequently a customer visits the company's website, giving us insight into their level of engagement.

Finally, we have preferred channel, which shows how the customer prefers to interact — whether that's through a mobile app, a website, or in-store.

To use these features effectively in models like K-Means, we need to prepare the data. Categorical features like region and channel must be encoded, and numerical features — such as age, income, and visit count — should be scaled. This ensures that all features are treated fairly during clustering, allowing the model to uncover meaningful customer segments based on the patterns it detects.

This task is not supervised learning — because we're not trying to predict a label or outcome.

In supervised learning, the goal is to answer specific questions like: "Will the customer churn?" or "Will they click on this ad?"

To do that, we need labelled data — where each example comes with the correct answer.

But in customer segmentation, we don't have any target labels.

We only have input features — like age, income, or visit count — and we ask the model to discover patterns or groupings based on that data.

This is what defines unsupervised learning — the model learns structure purely from input features, without being told what the correct answer is.

And because there are no labels, the way we evaluate results is also different. Instead of accuracy, we look at things like cluster separation, silhouette score, and business insights — focusing more on interpretability and pattern quality than numeric performance.

Once clustering is complete, we can start profiling the groups to understand what they represent.

For example, one group might include customers with high income but low spending. These customers have strong potential, but they may need more personalised offers



to encourage engagement.

Another cluster may be made up of young, high-spending customers — trendsetters who are more likely to respond to new product launches.

Then there's a third group with moderate income and moderate spending. These customers are consistent and reliable, making them ideal for loyalty and retention programmes.

By turning these segments into actionable profiles, businesses can build targeted strategies — from marketing campaigns to product placement — all driven by the patterns uncovered through unsupervised learning.

Anomaly detection is a powerful and widely used example of unsupervised learning. It's common in areas like security, finance, and healthcare.

The goal is to detect unusual data points — things that don't follow the normal pattern. These could be fraudulent transactions, equipment failures, or abnormal patient readings.

The challenge is that anomalies are rare, and in most real-world situations, they aren't labelled ahead of time. We don't know in advance which data points are normal and which ones are not.

That's why we use unsupervised learning. Instead of learning from known labels, the model learns what "normal" looks like based on the data it sees. Then, it flags anything that looks very different from that norm.

This lets us spot potential issues early, without needing a labelled dataset to begin with.

Let's explore how anomaly detection is used in the real world — across different industries — to catch unusual events early.

We begin in finance, where anomaly detection is a key tool for fraud prevention. By learning a customer's usual behaviour, the system can flag transactions that look out of place, like a sudden high-value purchase or a duplicate insurance claim.

In cybersecurity, it plays a different but equally important role — helping detect suspicious activity. For example, if there's a login from an unfamiliar location or at an odd time of day, the system marks it as unusual.

Moving into manufacturing, anomaly detection helps monitor machines. A sudden spike or drop in sensor readings could mean a fault is coming, allowing for early maintenance before the system fails.

In healthcare, the stakes are even higher. Here, anomaly detection might catch a sharp change in a patient's vitals — like heart rate or oxygen levels — signalling an urgent medical condition.

In the e-commerce space, the focus shifts to behaviour. The model can flag things like fake product reviews, or bulk orders that don't fit a customer's past history.

And finally, in banking, it helps detect money laundering by spotting patterns that break away from normal cash flow or transaction behaviour.

Across all these cases, the key point is this: anomalies are rare, and we usually don't have labelled examples to train on.

That's why we rely on unsupervised learning — so the model can learn what "normal" looks like, and flag anything that deviates too far from that pattern.



In unsupervised anomaly detection, the model's first task is to learn the structure of what's considered "normal".

There are no labels — no tag that says "fraud" or "not fraud" during training. The model must understand patterns in the majority of the data and define what normal looks like on its own.

Once that normal pattern is learned, the model can start spotting what doesn't fit. Any data point that falls too far outside the learned pattern is flagged as an anomaly.

There are a few different ways to do this. One approach uses distance-based models, such as D-B-SCAN or K-N-N, that flag outliers based on how far they are from other points.

Another method involves density-based models, like Isolation Forest or One-Class S-V-M, which detect regions in the data where very few points exist. These models assume that rare, isolated points are more likely to be anomalies.

A third approach is based on reconstruction models — for example, using autoencoders. These models try to recreate the input data. If a point can't be accurately reconstructed, it may be flagged as an anomaly.

That's what makes anomaly detection a classic unsupervised problem — because it's all about learning without labels, and using those patterns to find the unexpected.

This slide shows examples of features commonly used in anomaly detection, along with where they're applied and what type of data they represent.

We begin with transaction amount, which is a numerical feature used in financial fraud detection — helping flag unusual spending patterns.

Next is time of day, often used in behavioural modelling. Since time repeats every 24 hours, it's treated as a cyclical or numeric feature to capture recurring patterns.

Then we have device type or location, which helps detect bots or access from unusual places. This is a categorical feature, meaning it must be encoded before use.

In industrial settings, features like CPU temperature or machine R-P-M — that's revolutions per minute — help identify equipment faults. These are clearly numerical values.

Similarly, in healthcare, heart rate and blood pressure are used to detect medical anomalies. Again, these are numerical features that track deviation from normal physiological ranges.

And finally, session duration is used in web or network monitoring. It helps detect unusually long or short sessions, and is also treated as a numerical feature.

Choosing the right features — and knowing how to represent and process them — is key to helping models accurately detect what doesn't fit the norm.

Once anomalies are detected, the real value comes from how we act on them. Let's explore how this plays out across different domains.

In fraud detection, an unusual transaction doesn't just sit in the data — it triggers a fraud alert, prompting human review. This helps banks catch threats before they escalate.

Now shift to manufacturing. Here, small deviations in sensor readings might point to early signs of machine wear. That means predictive maintenance can happen before breakdowns occur — reducing costly downtime.



In cybersecurity, an unexpected login from an unusual location could signal a breach. Anomaly detection helps raise real-time alerts, strengthening system defences.

In healthcare, changes in vitals like heart rate or blood pressure might be early signals of illness or recovery. Spotting these outliers quickly supports better patient care.

And in risk management, sudden spikes or dips in time series data — like sales or usage — can indicate outliers worth deeper investigation.

So, across domains, anomaly detection transforms raw data into real-time, actionable intelligence — all without needing labelled examples in advance.

Document clustering is an unsupervised learning technique that groups similar text documents — based entirely on their content.

Here's the key idea: we don't provide any labels or categories in advance. Instead, the model looks at the words and patterns in each document to find natural groupings on its own.

The goal is to uncover hidden topics or themes — like customer complaints, technical questions, or positive feedback — across large volumes of text.

We use this approach when we don't have labelled data. Think of reviews, support tickets, or news articles — they rarely come pre-tagged.

That's where unsupervised learning becomes powerful. It helps us structure messy, unlabelled text into meaningful clusters — revealing insights without the need for manual categorisation.

Document clustering is used across many fields — especially when we have large volumes of unlabelled text and want to uncover useful patterns.

In news aggregation, articles are grouped by topic — like politics, sports, or technology. This helps automate tagging and supports personalised content delivery.

For customer feedback, clustering helps sort reviews by complaint type or praise. This gives teams a clearer picture of what users are saying without manually labelling each response.

In research, we can use clustering to discover common themes across published papers. This supports faster literature reviews and helps track emerging trends.

Legal teams use clustering to organise documents like contracts or filings by subject. It makes it easier to retrieve information and reduces manual sorting.

And in e-commerce, clustering product descriptions can improve how search results are ranked or how recommendations are made.

Across all these examples, one thing is constant — there are no predefined labels. Unsupervised learning helps us structure raw text and unlock insights we wouldn't easily see otherwise.

Let's walk through the process of clustering documents step by step.

We begin by converting raw text into numeric form. This is done using techniques like T-F I-D-F — that's term frequency-inverse document frequency — or word



embeddings. These help translate words into vectors that a machine can work with.

Next, we normalise and scale these vectors. This ensures that each feature contributes fairly to distance-based calculations.

Then, we reduce the dimensionality. Methods like P-C-A or t-distributed stochastic neighbour embedding — t-S-N-E — help simplify the data for better processing or visualisation.

Once the data is ready, we apply clustering algorithms such as K-Means, Agglomerative Clustering, or D-B-S-C-A-N to find natural groupings.

Finally, we interpret these clusters by looking at the most common words or themes in each group. This helps us understand the meaning of each cluster — even though no labels were given.

This whole process happens without any predefined labels. The algorithm uncovers structure based on the language itself — not on human input.

Documents don't come with built-in features — we create them from text.

We start by measuring basic word frequency using the Bag of Words method. Then we move to T-F-I-D-F, which highlights important terms by giving more weight to words that appear rarely but meaningfully.

To go beyond exact word matching, we use Word Two Vec or BERT, which capture semantic meaning based on context.

We can also use n-grams to capture common phrases instead of just individual words. And when needed, we add meta features like document length or readability to enrich the clustering process.

Once we've extracted these features, we represent them as numeric vectors — ready for clustering, just like any other structured dataset.

Once we have document clusters, we can generate powerful insights.

We start with auto-tagging — automatically assigning topics to documents or emails without manual effort.

Next is trend analysis, where we track how dominant themes shift over time, such as rising concerns in customer feedback.

In knowledge management, clusters help organise large collections of text — like research papers or internal documents — based on content similarity.

For customer voice, we group feedback into themes such as delivery, product quality, or pricing, helping teams understand user needs at scale.

And finally, search optimisation — clusters help return more relevant results by using document similarity to improve retrieval.

All of this is powered by unsupervised learning, enabling structured insights from messy, unlabelled text.

Now that you've seen how machines uncover patterns without labels, think about where else this could apply. What kinds of problems can we solve when we let data speak for itself?

Video 9.3 - Applications of Unsupervised Learning (02:43)

How do you think unsupervised learning will improve tumor detection in medical imaging?

Medical image segmentation is crucial in healthcare, dividing images into meaningful regions like tissues and organs. This precision is essential in radiology and oncology, guiding surgeries and treatment plans. For instance, a surgeon must accurately identify where a tumour begins and ends before operating.

However, medical images are complex, with high variability, noise, and low contrast, making segmentation challenging. Labelling images for supervised learning is slow, expensive, and requires expert knowledge. Radiologists would need to manually outline hundreds or thousands of images, creating barriers to developing effective AI models.

Unsupervised learning automates segmentation, eliminating the need for labelled data. This speeds up the process, reduces costs, and enables AI models to generalise across different medical centres and imaging devices, improving efficiency and patient care.

Unsupervised learning eliminates the need for labeled data by grouping pixels based on similarities like intensity and texture. Clustering techniques, such as K-means, automatically segment medical images without human input, distinguishing tissues and detecting subtle patterns. In MRI scans, these models can differentiate tumors from healthy tissue, achieving over **85% accuracy** in brain tissue segmentation.

Autoencoders further enhance accuracy by learning hidden structures in images. Studies show these methods improve **brain tumor segmentation by 15 to 20%** and increase **early breast cancer detection rates by up to 10%**. Recent reports highlight that **30% of AI-driven medical segmentation tasks** now use unsupervised learning, with adoption growing by over **50% from 2018 to 2022**.

Beyond improving accuracy, unsupervised learning reduces the need for manual labeling and adapts to various imaging modalities—making it a game-changer in healthcare.

How can unsupervised learning revolutionise medical imaging in the next decade?
Explore more!

Video 9.4 – Distance Metrics (19:35)

Hello and welcome everyone.

Let us get ourselves introduced to distance metrics in unsupervised learning.

Distance metrics are the fundamental tools in unsupervised learning, particularly in clustering and dimensionality reduction tasks.

Unlike supervised learning where models are trained using labeled data to predict outcomes, you have seen that unsupervised learning focuses on finding patterns and structures within unlabelled data, and this is possible by determining the similarity or dissimilarity between the observations.

Also, since we have multiple distance metrics defined, the choice of which metric significantly affects the performance of various unsupervised learning algorithms, which



you will see like the K means, the hierarchical clustering, nearest neighbors, etcetera.

So let's try to understand what is a distance metric.

Distance metrics essentially measure how far apart data points are in a given space.

In a way, they transform the abstract concept of similarity into a tangible value, and that is something which can be used to group similar data points or identify structures within the data.

Some of the common distance metrics which are used in unsupervised learning is Euclidean distance.

The most intuitive widely used distance metric, and it's a very simple calculation using the straight line distance between 2 points in Euclidean space.

The next one is the Manhattan distance, also known as the L1 norm or Taxicab distance, and it measures the distance between 2 points by summing the absolute differences of their coordinates.

We will see a working example below.

This is useful when you are dealing with grid like paths or when the dimensions have different units.

The 3rd one is cosine similarity which measures the cosine of the angle between 2 vectors.

Very often used in text analysis, you would see that the values range from one to one where one indicates identical vectors and one indicates completely opposite vectors.

And this is very effective in situations where the magnitude of the vector is less important than their direction.

Now as we discussed that distance metrics allows algorithms to group similar points or identify outliers and in the K means clustering, you would know that the distance metric is going to help us to assign the points to the nearest cluster center.

Similarly in anomaly detection, you would use this to identify the points which are unusual and they deviate significantly from the rest of the data.

That is how anomaly is defined.

Let's try to understand that through some visuals and some working examples.

So we have a plot of 2 points A and B with their respective coordinates 23 and 71.

Since they are 2 coordinates, they would be plotted on a 2 dimensional plane.

Now the line joining them would represent the distance between the points.

Now how do we measure this distance?

Here comes Euclidean distance as we discussed the most commonly used distance metric and it measures the straight line.

If you remember this formula from your high school, this is the formula between 2 points in the form of a straight line $\sqrt{X_2^2 - X_1^2 + Y_2^2 - Y_1^2}$ the whole square plus $\sqrt{Y_2^2 - Y_1^2}$ the whole square, where X_1 and Y_1 are the coordinates of the point A and X_2 and Y_2 are the coordinates



point b in a 2 dimensional space.

Now if you go for a higher dimension, which is very usual in machine learning, then this formula could be generalised in this manner.

Well, we don't have to worry about that because Python takes care of scaling it in higher dimensions.

Let's see an example.

So for the example 2, 3 and 71, if I want to calculate the Euclidean distance, so I'm going to place the numbers appropriately.

So X_2 which is 7 -, X_1 which is 2 the whole square, plus Y_2 which is 1 -, Y_1 which is 3 the whole square.

And that will then be solved and you would see that you can solve that and you would get a sum of 29 and take a square root of that.

That would give you 5.39.

So if you want to calculate the distance between these 2 points, this is how you do the mathematical calculation about it.

Let's try to take a slightly different example and see it in the visual manner.

And for that I'm going to use Python.

You don't have to worry about the code over here.

We are more interested in the working in the explanation about how distances are calculated.

So we are taking 4 different points A, B, C & D with their respective coordinates and I'm just trying to plot that over here.

Again, as I said, you don't have to worry about these codes.

You can take your own time, you can try to explore that, but I'm more interested in understanding the inclusion behind distance matrix.

And these are the 4 different points.

Now if I ask you simply that, which are the 2 most closest points over here?

And if you carefully observe these 4 points, you would yourself be able to say that point C & D are closer apart compared to all other points.

And it looks like point C & B are farthest part.

But let's try to give them some values.

And that is where I'm going to use the Euclidean metric, which is the same formula which we have just seen above.

Now I'm going to pair all the points like create all the combinations.

I'll create combination of A with all the 3, with B with all the 2 and with C & D.



So I will have all the combinations over here.

That is what pairs will actually be able to generate for me.

Now for each pair, I am going to calculate the distance and I am going to plot that distance.

Now let me show you how the graph would look like and how it would validate your intuition earlier which you made.

So this is where you have the distances between all the points in the plane and if you see the smallest one of them is C to D, that's 2.24.

And that is exactly what we had hypothesised earlier.

And C to b is 6.71, which is the largest for all of them.

So whatever we saw, we observed, we are able to give them the value and the value validates our observation.

So this is a classic example of how Euclidean metric helps us identify the distances between 2 points.

Now moving next to Manhattan distance, as we discussed, it's also called as a taxi cab distance or the L1 distance.

It's a very simple calculation.

It says that you can measure the distance between 2 points by simply summing the absolute differences of their coordinates.

And this is what is called as Manhattan distance because it is what mimics the way distances are measured in a grid like path.

You move either horizontally or you move vertically.

You cannot move diagonally.

So the formula basically says for the same set of coordinates in a 2 dimensional plane, simply you take the absolute differences between their coordinates.

So $X_2 - X_1 + Y_2 - Y_1$.

Now in case if you go into the n dimensional space, the formula can be generalised, but again you don't have to worry about that, Python takes care of it.

Let's see with an example with the same set of points 23 and 71, we replace the values of X_2 and X_1 , Y_2 and Y_1 and we simply get an answer and the answer is 7.

Well, let's try to again visualise this using Python.

So I'm going to take the same idea.

I'm going to take the same set of points and I'm going to plot them.

This is where the plot looks like exactly the same as earlier.

Now we need to identify if I move if I'm only allowed to move vertically and horizontally, which are the 2 closest points.



What I'm trying to say is that you cannot move from C to D like this.

If you have to go from C to D, you have to follow this grid.

You have to go like this below and then you turn towards D. If you want to travel from D to B, it will follow this path and then you go to B.

What you cannot do is you cannot go from D to B directly diagonally because there is no path from D to B there.

There is no diagonal path available.

So that's what Manhattan calculates now.

Again, if I ask you a question that according to you which are the 2I would say which is the closest point, you would again say C&D are the closest ones.

And obviously C to B seems to be again, they are quite far apart.

Let's see what Manhattan calculates now.

Again, we are going to calculate the Manhattan.

If you see the formula, it's simply nothing but summing up the absolute values between the pairs.

I'm going to form the same set of pair as I did earlier, and I'm going to plot that.

So again, I don't want you to go into this code, right, because that's not required.

This is just for the sake of helping you understand what distance metrics mean, right?

If you're very interested, you can go through that, but it's not required.

So this is what we see.

And clearly if you calculate all the distances you see it's 7, this is 7, this is 636 and 9.

So the smallest one of all of them is C to D, which is 3 and B to C, which is 9 is the largest of all of them.

So again, it also validates our claim, which we made earlier, that C to D would be the shortest one and C to B would be the longest one.

So if you see the difference between Manhattan and Euclidean is in Manhattan, you are not moving diagonally, you're only moving along the grid.

That's the difference.

Well, the next distance is the cosine distance.

Very interesting distance metric.

When you are trying to measure the direction of the vectors in the similarity in terms of the direction of vectors.

Basically you will use the angle between the vectors.



So for text analysis, for document similarity task, you will see that we use a lot of cosine distance.

If you know about open EI embeddings, if you want to calculate the similarity between 2 paragraphs, 2 documents, you actually convert them to embeddings using these generative EI models.

And then you calculate the cosine distance between those embeddings to find out that which 2 documents or which 2 paragraphs are close.

So it is this distance metric which is used over there.

So let's try to understand what this distance metric is about.

And if you go back and recall, there was a technique called as dot product.

So this metric uses dot product.

If you remember the formula of the dot product, $a \cdot b$ was magnitude of a times magnitude of b $\cos \theta$.

So I'm just using that and reordering it.

I'm saying the $\cos \theta$, that is the cosine similarity is nothing but the dot product divided by the multiplication of their magnitudes.

So that's basically explained over here.

Now that is my cosine similarity.

But if I want to find cosine distance, I will subtract one cosine similarity.

So this is something which is very important for you to remember.

Cosine distance is one cosine similarity.

So let's try to see for the same set of example 23 and 7, I'll take a dot product.

So I'll multiply their corresponding coordinates.

So X_1 is multiplied with X_2 .

So $27 \times Y_2$ is Y_1 is multiplied with Y_2 .

That's $313 \times X_1$ and you calculate that you get the number 70.

This is the formula to calculate the magnitudes.

So you take their coordinates and take a square root of the sum of their squares.

So because A was 23, you are going to do it 2^2 plus 3^2 which is 49, which is root of 49 which is around 7, almost similar.

You will do it for P also and you get a number 7.07.

Now you have all the numbers.

You can just replace these numbers in this formula and you can try to find out the cosine similarity.



So let's try to do that.

So the cosine similarity, I just place those numbers and I get a number 0.67.

And as I said earlier, you have to subtract this number from one to get the cosine distance.

So which is what you get as 0.33, which is telling me that the vectors are somewhat similar in direction.

33 is not a very great number, but fine.

You know it tells that there is a tendency to be similar.

Let's try to do that in Python.

Now for that instance, I have taken the same set of points and I am going to plot those points as position vectors.

Now if I ask you that which 2 vectors have a very small deviation between them, the smallest angle between them.

So you would clearly say that position vector A and position vector C the angle between them is the smallest compared to any other pair of vectors.

And similarly position vector C and position vector B where are their their distance apart in terms of their deviation, they have a large angular value.

Let's try to measure that using the formula which we have seen above.

Again, Python takes a lot of these tasks for South and and performs.

That would leave that up to Python for that to do.

But if I see the similarity between the vectors, you see that A and C they are 0.99 almost one.

And remember one is almost similar.

Whereas the point C&B, if you calculate the distance between C&B, you will see that it is 0.57, the lowest of all of them.

And then the other values are also advocating that how similar the 2 vectors are.

So these are some important distance metrics and we understand the intuition between the 3 different metrics.

You use Euclidean if you are allowed to move diagonally, you use Manhattan if you are not allowed allowed to use diagonally and then you go to cosine.

If you are interested in vector similarity that is in any data represented in the form of vectors, you will use cosine similarity over there.

Well, that's all from my side for distance metrics and I hope that whatever we have learned today would become very useful in the next few in the next few sessions on K means hierarchical clustering and other algorithms like anomaly detection.

Thank you everyone.